The Overclocking Algorithm (OCA) - Detailed Notes

Course/Lab: MHO Lab
Date: March 2026

Table of Contents

1. Introduction
2. Characteristics of OCA
3. Flowchart
4. Initialization
5. Detailed Algorithm
6. Termination Criteria
7. User-Defined Parameters
8. Pseudocode

---

1. Introduction

Metaheuristic optimization algorithms are indispensable tools for solving complex, nonlinear, multimodal, and non-convex optimization problems where deterministic or gradient-based methods are either infeasible or computationally prohibitive. While population-based methods such as Particle Swarm Optimization (PSO), Grey Wolf Optimizer (GWO), Genetic Algorithms (GA), and Differential Evolution (DE) have achieved widespread adoption, they frequently encounter a fundamental tradeoff: algorithms that are accurate on difficult landscapes tend to be slow, while fast algorithms may lack the diversity mechanisms needed to escape local optima.

The **Overclocking Algorithm (OCA)** addresses this gap by drawing inspiration not from biological systems, but from the architecture and runtime behavior of modern Central Processing Units (CPUs). A CPU continuously balances clock frequency, voltage, thermal output, and instruction throughput - all within strict physical constraints. OCA translates these engineering principles into mathematical operators for global optimization.

Formal objective:

$$\mathbf{x}^* = \arg \min_{\mathbf{x} \in [LB, UB]^d} f(\mathbf{x})$$

where f is the objective function, d is the dimensionality, and $[LB, UB]$ are the lower and upper bounds of the decision variables.

1.1 The CPU-Optimization Metaphor

CPU Concept	Optimization Equivalent
CPU Core	Search Agent (candidate solution)
Throughput (IPC)	Fitness Value
P-Cores (Performance Cores)	Elite Leaders (P_1, P_2, P_3)
Voltage	Exploration Step Size
Throttling	Convergence / Exploitation
Cache Miss	Stagnation Detection and Reset
Instruction Pipeline	Velocity / Momentum

---

2. Characteristics of OCA

The Overclocking Algorithm possesses the following defining characteristics:

1. **CPU Metaphor Mapping**
   Every major operator in OCA maps to a well-understood CPU concept (core, voltage, throttling, cache miss, pipelining), making the algorithm intuitive to describe and reason about.

2. **Tri-Core Leadership**
   Instead of relying on a single global best (as in PSO) or a single alpha (as in basic GWO), OCA maintains the top 3 best solutions found so far, designated as Performance Cores P_1, P_2, and P_3. Each agent's target position is the average of suggestions from all three leaders, reducing sensitivity to any single elite solution.

3. **Dynamic Voltage and Frequency Scaling (DVFS)**
   A linearly decaying "voltage" parameter controls the amplitude of exploration. Early iterations operate at high voltage (wide search), and later iterations throttle down to low voltage (fine-grained exploitation). This mirrors the Turbo Boost behavior of real CPUs.

4. **Instruction Pipelining (Momentum)**
   Each agent carries a velocity vector that accumulates direction over iterations. This momentum term accelerates progress through narrow valleys (e.g., Rosenbrock) and reduces oscillatory behavior near optima.

5. **Cache Miss Reset**
   A per-agent stagnation counter tracks consecutive iterations without meaningful improvement. When the counter exceeds a threshold, the agent is "flushed" and reinitialized near a randomly selected elite core with additive noise. This mechanism is critical for escaping deceptive local optima (e.g., Schwefel).

6. **Vectorized (Lite) Design**
   OCA V7 (Lite Edition) formulates all update equations as element-wise vector operations, avoiding nested loops common in standard GWO implementations. This design yields practical speedups of approximately 3x to 6x over GWO.

---

3. Flowchart

```
START
  |
  v
Set parameters: N, d, T_max, LB, UB, stall_limit, epsilon, eta, sigma
  |
  v
Initialize population X[1..N] and velocity Vel[1..N]
  |
  v
Evaluate fitness f(X[i]) for all agents
  |
  v
Select leaders P1, P2, P3 <- top-3 solutions by fitness
  |
  v
+----------- For t = 1 to T_max -----------+
|                                           |
|  1. Compute DVFS voltage V(t)             |
|  2. For each agent i:                     |
|     a. Compute coefficients A, C          |
|     b. Compute D_k, Step_k for k=1,2,3    |
|     c. Compute X_target (average of Steps) |
|     d. Update velocity Vel[i]             |
|     e. Update position X[i]               |
|     f. Apply boundary handling            |
|     g. Evaluate fitness                   |
|     h. Update stall counter               |
|     i. If stall > threshold -> cache-miss   |
|        reset near random P-Core           |
|  3. Update leaders P1, P2, P3             |
|  4. Check termination criteria            |
|                                           |
+-------------------------------------------+
  |
  v
Return best solution P1 and fitness f(P1)
```

```
  |
  v
END
```

---

4. Initialization

4.1 Population Initialization

A population of N agents (cores) is generated uniformly at random within the search space:

$$\mathbf{X}_i^{(0)} = \mathbf{LB} + \mathbf{r}_i \odot (\mathbf{UB} - \mathbf{LB}), \quad i = 1, \ldots, N$$

where $\mathbf{r}_i \sim U(0,1)^d$ is a d-dimensional random vector drawn from a uniform distribution, and $\odot$ denotes element-wise (Hadamard) multiplication.

4.2 Velocity Initialization

All velocity vectors are initialized to zero:

$$\mathbf{Vel}_i^{(0)} = \mathbf{0}, \quad i = 1, \ldots, N$$

4.3 Stall Counter Initialization

Each agent has a stagnation counter initialized to zero:

$$stall_i^{(0)} = 0, \quad i = 1, \ldots, N$$

4.4 Fitness Evaluation and Leader Selection

The fitness of every agent is computed:

$$Fit_i^{(0)} = f\left(\mathbf{X}_i^{(0)}\right)$$

The three agents with the best fitness values are stored as the elite leader registry:

$$P_1, P_2, P_3 = \text{top-3 solutions sorted by fitness}$$

---

5. Detailed Algorithm

5.1 Dynamic Voltage and Frequency Scaling (DVFS)

At each iteration t, a voltage parameter controls the exploration range:

$$V(t) = 2\left(1 - \frac{t}{T_{max}}\right)$$

- At $t = 0$: $V = 2$ (maximum exploration - "Turbo Boost").
- At $t = T_{max}$: $V = 0$ (pure exploitation - "Throttled").

5.2 Coefficient Vectors

For each agent i at iteration t, two stochastic coefficient vectors are computed:

$$\mathbf{A}_i(t) = 2V(t)\mathbf{r}_{1,i} - V(t)$$

$$\mathbf{C}_i(t) = 2\mathbf{r}_{2,i}$$

where $\mathbf{r}_{1,i}, \mathbf{r}_{2,i} \sim U(0,1)^d$.

- When $|A| > 1$: the agent explores regions far from the leaders.
- When $|A| < 1$: the agent converges toward the leaders.

5.3 Distance and Candidate Steps from Elite Leaders

For each elite leader $k \in \{1, 2, 3\}$, a weighted distance and a candidate movement step are computed:

$$\mathbf{D}_{ik}(t) = |\mathbf{C}_i(t) \odot \mathbf{P}_k(t) - \mathbf{X}_i(t)|$$

$$\mathbf{Step}_{ik}(t) = \mathbf{P}_k(t) - \mathbf{A}_i(t) \odot \mathbf{D}_{ik}(t)$$

Each $\mathbf{Step}_{ik}$ represents the position an agent would move to if guided solely by leader P_k.

5.4 Tri-Core Target Aggregation

The final target position for agent i is the average of the three candidate steps, ensuring balanced influence from all elite cores:

$$\mathbf{X}_{target,i}(t) = \frac{1}{3}\sum_{k=1}^{3} \mathbf{Step}_{ik}(t)$$

5.5 Instruction Pipelining - Momentum Update

To carry forward successful movement direction and prevent oscillation, OCA applies a momentum (velocity) update analogous to instruction pipelining in a CPU:

$$\mathbf{Vel}_i(t+1) = w_i(t)\,\mathbf{Vel}_i(t) + (\mathbf{X}_{target,i}(t) - \mathbf{X}_i(t))$$

$$\mathbf{X}_i(t+1) = \mathbf{X}_i(t) + \eta\,\mathbf{Vel}_i(t+1)$$

where: - $w_i(t) \sim U(0.5, 0.9)$ is a randomly sampled inertia weight. - $\eta = 0.5$ is the velocity scaling factor.

The randomized inertia weight introduces stochasticity into the momentum term, preventing all agents from converging along the same trajectory.

5.6 Boundary Handling

After position update, each agent is clamped to the feasible search space:

$$\mathbf{X}_i(t+1) = \min\big(\max(\mathbf{X}_i(t+1), \mathbf{LB}), \mathbf{UB}\big)$$

5.7 Fitness Evaluation and Improvement Tracking

The new fitness is computed:

$$Fit_i(t+1) = f(\mathbf{X}_i(t+1))$$

The improvement magnitude is checked:

$$\Delta f_i(t) = f(\mathbf{X}_i(t)) - f(\mathbf{X}_i(t+1))$$

- If $\Delta f_i(t) > \epsilon$: $stall_i \leftarrow 0$ (meaningful improvement detected).
- If $\Delta f_i(t) \leq \epsilon$: $stall_i \leftarrow stall_i + 1$ (stagnation detected).

5.8 Cache Miss Reset

When an agent's stall counter exceeds the user-defined threshold, a cache-miss event is triggered:

$$stall_i > stall_{limit} \implies \text{Cache Miss for agent } i$$

The agent is reinitialized near a randomly selected elite core with Gaussian noise:

$$\mathbf{X}_i \leftarrow \mathbf{P}_r + \sigma \mathcal{N}(0, \mathbf{I})$$

where P_r is chosen uniformly at random from $\{P_1, P_2, P_3\}$ and σ controls the perturbation scale. The stall counter is then reset:

$$stall_i \leftarrow 0$$

This mechanism ensures that stagnated agents are recycled into the search process with fresh starting positions near known high-quality regions, significantly improving performance on deceptive landscapes like Schwefel.

5.9 Leader Update

After all agents have been updated, the elite leader registry is refreshed:

$$\{P_1, P_2, P_3\} \leftarrow \text{best-3 solutions in the current population}$$

---

6. Termination Criteria

The algorithm terminates when **any one** of the following conditions is satisfied:

6.1 Maximum Iterations (Primary)

$$t \geq T_{max}$$

This is the default and always-active stopping condition.

6.2 Target Fitness Achieved (Optional)

$$f(P_1) \leq f_{target}$$

If a known global optimum or acceptable fitness threshold exists, the algorithm can stop early once this target is reached.

6.3 Global Convergence Stagnation (Optional)

$$|f_{best}(t) - f_{best}(t - L)| < \delta$$

where L is a patience window (number of iterations to look back) and δ is a tolerance. If the global best fitness has not improved by more than δ over the last L iterations, the search is considered converged.

---

7. User-Defined Parameters of OCA

Parameter	Symbol	Description	Typical Value / Range
Population size	N	Number of search agents (cores)	20 - 60
Dimensionality	d	Number of decision variables	Problem-dependent
Max iterations	T_{max}	Upper limit on optimization iterations	100 - 1000
Variable bounds	LB, UB	Lower and upper bounds of the search space	Problem-dependent
Inertia weight	w	Momentum scaling factor (sampled per agent per iteration)	$U(0.5, 0.9)$
Velocity scaling	η	Fraction of velocity applied to position update	0.5
Stall threshold	$stall_{limit}$	Consecutive non-improving iterations before cache-miss reset	8 - 15
Improvement tolerance	ϵ	Minimum fitness change to count as improvement	10^{-8} - 10^{-4}
Reset noise scale	σ	Standard deviation of Gaussian noise during cache-miss reset	0.01 - 0.2 x search span
Target fitness	f_{target}	Optional early-stop fitness threshold	User-defined (optional)
Patience window	L	Iterations lookback for global stagnation check	User-defined (optional)

Parameter	Symbol	Description	Typical Value / Range
Stagnation tolerance	δ	Minimum global best change over patience window	User-defined (optional)

Notes: - N, T_{max}, LB, UB, $stall_{limit}$, ϵ, η, and σ are required. - f_{target}, L, and δ are optional early-stopping parameters. - The inertia weight w is not fixed but sampled from $U(0.5, 0.9)$ independently for each agent at each iteration, so it does not need manual tuning.

---

8. Pseudocode of OCA

```
ALGORITHM: Overclocking Algorithm V7 (Lite Edition)

INPUT:
    f           -  Objective function
    N           -  Population size
    d           -  Dimensionality
    LB, UB      -  Lower and upper bounds (d-dimensional)
    T_max       -  Maximum number of iterations
    stall_limit -  Cache-miss trigger threshold
    epsilon           -  Improvement tolerance
    eta           -  Velocity scaling factor (default 0.5)
    sigma           -  Reset noise scale

OUTPUT:
    P1          -  Best solution found
    f(P1)       -  Best fitness value

BEGIN
    // -- Initialization --
    FOR i = 1 TO N DO
        X[i]     <- LB + rand(d) * (UB - LB)
        Vel[i]   <- zeros(d)
        stall[i] <- 0
        fit[i]   <- f(X[i])
    END FOR

    P1, P2, P3 <- top-3 solutions from X sorted by fitness

    // -- Main Loop --
    FOR t = 1 TO T_max DO
```

```
// DVFS voltage
V <- 2 x (1 - t / T_max)

FOR i = 1 TO N DO
    // Stochastic coefficients
    r1, r2 <- rand(d), rand(d)
    A <- 2 x V x r1 - V
    C <- 2 x r2

    // Distance and step from each P-Core
    FOR k = 1, 2, 3 DO
        D_k  <- |C * P_k - X[i]|
        Step_k <- P_k - A * D_k
    END FOR

    // Tri-Core target
    X_target <- (Step_1 + Step_2 + Step_3) / 3

    // Momentum (instruction pipelining)
    w <- uniform(0.5, 0.9)
    Vel[i] <- w x Vel[i] + (X_target - X[i])
    X_new  <- X[i] + eta x Vel[i]

    // Boundary handling
    X_new <- clip(X_new, LB, UB)

    // Fitness evaluation and stall tracking
    fit_new <- f(X_new)
    IF fit[i] - fit_new > epsilon THEN
        stall[i] <- 0
    ELSE
        stall[i] <- stall[i] + 1
    END IF

    // Cache-miss reset
    IF stall[i] > stall_limit THEN
        P_r   <- random choice from {P1, P2, P3}
        X_new <- P_r + sigma x normal(0, I_d)
        X_new <- clip(X_new, LB, UB)
        fit_new <- f(X_new)
        stall[i] <- 0
    END IF

    // Accept new position
    X[i]   <- X_new
```

```
            fit[i] <- fit_new
        END FOR

        // Update leader registry
        P1, P2, P3 <- top-3 solutions from X sorted by fitness

        // Optional early termination
        IF f(P1) <= f_target THEN BREAK
        IF |f_best(t) - f_best(t - L)| < delta THEN BREAK

    END FOR

    RETURN P1, f(P1)
END
```

End of document.